



RTL Design Sherpa

Pre-Bitstream Timing Prediction on FPGA Artix-7 Companion to the ASIC Methodology White Paper

June 12, 2026

Table of Contents

1 Readme Fpga.....	5
2 Pre-Bitstream Timing Prediction on FPGA (Artix-7).....	5
2.1 Abstract.....	5
2.2 1. What changes between the ASIC paper and this one.....	6
2.3 2. The harness on FPGA.....	7
2.4 3. The Vivado flow.....	7
2.5 4. The spreadsheet.....	8
2.5.1 4.1 characterization.....	8
2.5.2 4.2 building blocks.....	9
2.6 5. The validation loop.....	10
2.6.1 5.1 Wrapper.....	10
2.6.2 5.2 Signature accumulator.....	10
2.6.3 5.3 On-board observation.....	10
2.6.4 5.4 Vivado STA contract.....	11
2.6.5 5.5 Quick command.....	11
2.7 6. Recommended workflow.....	12
2.8 7. Limitations / intentional non-features.....	12
2.9 8. Navigation.....	13

List of Figures

No figures in this document.

List of Tables

No tables in this document.

1 Readme Fpga

Generated: 2026-06-12

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

2 Pre-Bitstream Timing Prediction on FPGA (Artix-7)

The FPGA companion to the ASIC pre-synth methodology. Same harness, same nine FUBs, same spreadsheet shape – but the lookup numbers come from Vivado primitive datasheets (and eventually from real post-route reports off the Nexys A7), and the validation loop runs through an on-board clock sweep instead of an STA tool.

Version: 0.1 (initial publication; calibration runs pending) **Last Updated:** 2026-06-12 **Companion to:** README.md (ASIC / ASAP7 paper) **Status:** Vivado flow live; spreadsheet seeded with datasheet numbers; hardware validation loop planned, not built.

2.1 Abstract

This document carries the same purpose as the ASIC paper: **predict and close timing before the bitstream build ever starts**. On ASIC, that prediction is grounded in Yosys + ABC + OpenSTA per-gate-delay measurements. On FPGA the inputs are different but the workflow is the same:

- characterize the relevant Xilinx primitives (LUT6, CARRY4, DSP48, BRAM, MUXF7/F8, LUTRAM, SRL) at the speed grade you'll ship at,
- record a per-primitive ps-per-level row on the characterization sheet,
- the building-blocks sheet looks those numbers up via VLOOKUP and reports a slack column for every block you might instantiate,
- a designer at the spreadsheet sees fail / pass per block as they tweak the planned clock frequency, **before** committing to a 10-30 minute Vivado place-and-route round trip.

The deliverable is `work/artix7_characterization.xlsx`, generated by `work/build_fpga_characterization.xlsx.py`. The Vivado flow lives under `fpga/` – a

Nexys A7-100T project that builds the same `char_top` harness, wrapped for a single board clock and eight observation LEDs.

This methodology is **explicitly proactive**. It is not a debugger to reach for after Vivado throws a negative WNS at you; it is a design-time budget table the designer consults *before writing the first `always_ff`*. The point is to flip the loop – characterize once per (device, speed grade), then build with a known-good budget – so place and route becomes a confirmation step rather than the first time anyone finds out the pipeline doesn't close.

2.2 1. What changes between the ASIC paper and this one

Topic	ASIC paper	FPGA paper
Process	TSMC-class ASAP7 PDK (open-source)	Xilinx Artix-7 (xc7a100tcsfg324-1 on Nexys A7-100T)
Toolchain	Yosys + slang + ABC + OpenSTA	Vivado (synth + impl + bitgen)
Corners	TT / FF / SS (Liberty PVT corners)	Speed grade -1 / -2 / -3
Frequency range	2.0 GHz - 3.0 GHz (where 7nm starts to choke)	100 - 300 MHz (where Artix-7 typically choke)
Per-level primitives	NAND, XOR, MUX, CARRY, MULT, GRAY_CTR, QUEUE	LUT6, MUXF7, MUXF8, CARRY4, DSP48, LUTRAM, SRL
Mixed-bag proxy	MULT (multiplier tree)	DSP48 (DSP macro)
Reference single-point	INV, CLK_DIV	BRAM (RAMB36 sync read)
Tflop source	Tcq + Tsu derived from NAND chain probe	FDRE Tcq + Tsu (~400 ps - 1, ~280 ps -3)
Wire delay model	per-metal-layer ps/mm (M2/M3 ... M8/M9)	single FPGA-routing ps/hop (default 200 ps -1)
Ground truth	ABC stime - c per chain	Vivado post-route report_timing_summary
Validation loop	corner data already there (same Liberty)	on-board MMCM clock sweep + host-side golden model

The spreadsheet's **building blocks** sheet is identical RTL with FPGA- specific numbers swapped in. The **stream** sheet (per-FUB port walks) is intentionally omitted from this v0.1 – the per-port hand-walks port over 1:1 from the ASIC sheet, only the looked-up per-level numbers change.

2.3 2. The harness on FPGA

char_top is the same nine-FUB harness described in [README.md Section 5](#). All anti-optimisation defences (LFSR-driven inputs, outputs routed to top-level ports, dont_touch attributes) work identically under Vivado.

For board-level use a thin wrapper trims the wide observation ports down to something the Nexys A7's 8 user LEDs can drive:

- fpga/rtl/char_top_fpga.sv instantiates char_top with all 9 FUBs enabled and XOR-reduces each FUB's output bus to a single bit.
- 8 LEDs drive one bit per FUB; the 9th FUB folds into LED[0] via an XOR with the NAND output so both chains reach a top-level port without losing an LED.
- A 100 MHz onboard oscillator (board pin **E3**) feeds clk; the CPU_RESETN button (**C12**) feeds rst_n; four switches (**J15** / **L16** / **M13** / **R15**) provide the LFSR seed nibble.

Every chain still reaches an output port, so Vivado optimisation can't delete anything – the FPGA equivalent of the ASIC scaffold.

2.4 3. The Vivado flow

The flow lives under fpga/, shaped like stream_characterization/flows-stream-bridge:

```
fpga/
├─ Makefile                                project | synth | bitstream |
program | clean
├─ rtl/char_top_fpga.sv                    thin wrapper around char_top
├─ constraints/char_top_fpga.xdc           Nexys A7-100T pin map + 100 MHz
clock
├─ tcl/
│   └─ create_project.tcl                  project gen from
rtl/filelists/char_top.f
│   └─ synth_only.tcl                      synth + utilization_synth.txt
```

```

├─ build_all.tcl          synth + impl + bitgen + reports
├─ program_fpga.tcl      JTAG flash via Vivado hw_server
└─ filelist_utils.tcl    env-var + +incdir + nested -f
support

```

Quick start (with REPO_ROOT exported via the repo's env_python):

```

cd projects/NexysA7/timing_characterization/fpga
make help          # see all targets
make synth         # ~2 min, posts utilization + post-synth timing
make bitstream     # ~10-30 min, full flow + all reports
make utilization   # cat the latest utilization report
make timing        # cat the latest post-route timing summary
make program       # flash if board is plugged in

```

Reports land under fpga/reports/:

File	Contents
utilization_synth.txt	post-synth LUT/FF/BRAM/DSP totals
utilization_impl.txt	post-route same
timing_summary.txt	per-path-group WNS, the calibration source
timing_worst.txt	top 50 worst paths
power.txt	Vivado's power estimate

The bitstream itself lands at fpga/bitstream/timing_char.bit.

2.5 4. The spreadsheet

work/artix7_characterization.xlsx is structured exactly like the ASIC companion. Two sheets:

2.5.1 4.1 characterization

Per-primitive ps-per-level rows for the seven primitives that scale with chain depth (LUT6, MUXF7, MUXF8, CARRY4, DSP48, LUTRAM, SRL), plus a single-point reference for BRAM access time. Speed grades -1 / -2 / -3 across the five-frequency sweep (100 / 150 / 200 / 250 / 300 MHz).

Sample TT-like numbers (Artix-7 -1 ballparks):

primitive	per-level ps (-1)	notes
LUT6	200	single-LUT propagation
MUXF7	250	F7 cascade
MUXF8	280	F8 cascade
CARRY4	80	per-bit CIN -> COUT (dedicated chain)
DSP48	2500	PROXY for mixed-bag logic
LUTRAM	600	distributed RAM read
SRL	400	SRL16 / SRL32 shift
BRAM (ref)	2400	RAMB36 sync read, 1-cycle

Tflop (FDRE Tcq + Tsu): 400 ps at -1, 340 ps at -2, 280 ps at -3.

The **Design clocks table** at the bottom is the single planning point. One row per clock the design uses:

clock name	freq (MHz)	target sg	period	input 60%	output 40%	wire/hop
clk_main	100	sg-1	derived	derived	derived	200
(5 empty slots)						

Yellow cells are editable; everything else derives. The “wire/hop” column defaults higher than the ASIC sheet (200 ps -1) because FPGA routing dominates timing more than ASIC standard-cell routing.

2.5.2 4.2 building blocks

One row per reusable RTL block, with editable params and corner-aware target columns:

- gaxi_skid_buffer
- gaxi_fifo_sync (REG=0)
- gaxi_fifo_sync (REG=1, BRAM-backed)
- arbiter_round_robin
- axi4_master_rd
- axi4_master_wr

- BRAM (raw RAMB36)

Each row's slack cell pulls from the Design clocks table via VLOOKUP – change `clk_main`'s freq or speed grade and slack flips per block. Negative slack renders **bold red**, three-decimal precision (0.000).

2.6 5. The validation loop

The ASIC paper has STA as the ground truth – Yosys + ABC + OpenSTA and the delay numbers come out of `stime -c`. On FPGA the equivalent is Vivado's post-route `report_timing_summary`, which `make bitstream` already emits. The on-board sanity check that “does the spreadsheet match what the chip actually does at this clock?” is the **MMCM sweep**:

2.6.1 5.1 Wrapper

`fpga/rtl/char_top_fpga_mmcm.sv` wraps an `MMCME2_BASE` primitive with a VCO running at 1200 MHz, exposing four test clocks to four parallel `char_top` instances:

domain	divide	test freq
<code>clk_test_0</code>	/8	150 MHz
<code>clk_test_1</code>	/6	200 MHz
<code>clk_test_2</code>	/5	240 MHz
<code>clk_test_3</code>	/4	300 MHz

Each instance gets the same LFSR seed nibble from `SW[1:0]`, so all four domains are doing the same work – just at different clocks. Per-domain reset is held until the MMCM locks, then released through a 2-flop synchroniser into each test clock.

2.6.2 5.2 Signature accumulator

`fpga/rtl/sig_accum.sv` folds every `char_top` output bus (207 bits worth – nand, inverter, xor, carry, mult, mux, queue, `clk_div`, gray) into a 32-bit XOR signature in the source domain. A free-running snapshot register updates every 2^{16} source-clock cycles and the value crosses into the 100 MHz observation domain via a 2-flop CDC synchroniser (Xilinx `ASYNC_REG = "TRUE"` attributes preserve the metastability margin).

2.6.3 5.3 On-board observation

Eight user LEDs show:

LED	meaning
<code>LED[0]</code>	<code>clk_test_0</code> (150 MHz) signature

LED	meaning
	changed since last snapshot
LED[1]	clk_test_1 (200 MHz) signature changed
LED[2]	clk_test_2 (240 MHz) signature changed
LED[3]	clk_test_3 (300 MHz) signature changed
LED[7:4]	lower 4 bits of the signature for the selected clock

SW[3:2] selects which domain drives LED[7:4] for closer inspection.

A failing clock domain's signature freezes (LED stays solid one way or the other instead of flickering at a few Hz), so a quick glance at LED[3:0] tells you which test clocks the chip is keeping up with.

2.6.4 5.4 Vivado STA contract

fpga/constraints/char_top_fpga_mmcm.xdc declares each MMCM output as a generated clock and groups them as asynchronous to the 100 MHz observation clock, so make_bitstream_mmcm produces a timing_summary.txt with one WNS row per clk_test_N group. Those WNS numbers are the calibration source: feed them back into work/build_fpga_characterization_xlsx.py's PRIM_PS dict and regenerate the workbook.

2.6.5 5.5 Quick command

```
cd projects/NexysA7/timing_characterization/fpga
make bitstream-mmcm    # builds char_top_fpga_mmcm with 4 test clocks
make program           # flash the board
                        # watch LED[3:0] for clock-domain liveness
```

What the v0.1 sweep does NOT include yet:

- **Host-side golden model.** The signature is structural (“did the chip keep producing changing data?”), not functional (“did the chip produce THE RIGHT data?”). A UART or USB-blaster handshake out to the host’s LFSR reference model would close that gap.
- **Automated calibration.** Reading timing_summary.txt and writing the per-LUT / per-CARRY4 numbers back into the spreadsheet builder is still a copy-paste workflow; a small script could automate it.

2.7 6. Recommended workflow

1. **Plan.** Open `work/artix7_characterization.xlsx`. On characterization, set `clk_main freq` + `speed grade` to your target. Flip to building blocks – any red rows are the blocks you’ll need to pipeline harder or use sparingly.
2. **Iterate the plan.** Lower the freq until everything’s green (the design’s actual ceiling on this device); switch the speed grade to model a faster-graded part; bump wire/hop to model longer inter-CLB routes.
3. **Build.** `cd fpga; make synth` for a quick utilisation check; `make bitstream` for the full place-and-route + reports.
4. **Calibrate.** Read `reports/timing_summary.txt` after `make bitstream`. Compare the per-FUB WNS against what the spreadsheet predicted. Update `work/build_fpga_characterization.xlsx.py`’s `PRIM_PS` dict with the calibrated numbers and regenerate the workbook.

The methodology pays for itself the first time it tells you “your plan doesn’t fit at 300 MHz on -1 silicon” before you spend 30 minutes on a Vivado run to learn the same thing.

2.8 7. Limitations / intentional non-features

- **Seed data are datasheet ballparks**, not calibrated measurements. Treat the v0.1 workbook as a methodology demo; the per-LUT / per-CARRY4 numbers are reasonable to start a discussion but should be replaced with calibrated values from real Vivado runs.
 - **Wire delay is one number.** ASIC has four metal layers in the workbook; FPGA’s interconnect is more uniform and the inter-CLB routing dominates. One ps/hop with a per-clock override is enough resolution for a SWAG.
 - **No MMCM sweep, no host golden model.** The on-board validation loop in section 5 is sketched but not wired up.
 - **No power numbers.** Vivado emits `report_power`; the workbook doesn’t yet model it.
-

2.9 8. Navigation

- ASIC companion paper: `README.md`
- Vivado flow Makefile: `fpga/Makefile`

- Board pin map: `fpga/constraints/char_top_fpga.xdc`
 - FPGA wrapper RTL: `fpga/rtl/char_top_fpga.sv`
 - FPGA spreadsheet: `work/artix7_characterization.xlsx`
 - Spreadsheet generator: `work/build_fpga_characterization.xlsx.py`
 - Components area: `../../components/README.md`
 - Repository guide: `/CLAUDE.md`
-

Version: 0.1 **Last Updated:** 2026-06-12 **Maintained By:** RTL Design Sherpa Project